

TELCO PROJECT SQL QUERY SOLUTIONS & APPROACH DOCUMENTATION

1.1 List the customers who are subscribed to the 'Kobiye Destek' tariff.

```
SELECT
    c.CUSTOMER_ID,
    c.NAME,
    c.CITY,
    c.SIGNUP_DATE,
    t.NAME AS TARIFF_NAME,
    t.MONTHLY_FEE
FROM
    CUSTOMERS c
    JOIN TARIFFS t ON c.TARIFF_ID = t.TARIFF_ID
WHERE
    t.NAME = 'Kobiye Destek'
ORDER BY
    c.NAME;
```

	123 CUSTOMER_ID	AZ NAME	AZ CITY	SIGNUP_DATE	AZ TARIFF_NAME	123 MONTHLY_FEE
1	425	Abdullah	BALIKESIR	2026-02-04 00:00:00.000	Kobiye Destek	350
2	8,383	Abdullah	TOKAT	2026-03-02 00:00:00.000	Kobiye Destek	350
3	290	Abdullah	ERZURUM	2025-11-06 00:00:00.000	Kobiye Destek	350
4	111	Abdullah	KIRIKKALE	2026-04-03 00:00:00.000	Kobiye Destek	350
5	140	Abdullah	İZMİR	2026-03-09 00:00:00.000	Kobiye Destek	350
6	9,891	Abdullah	KASTAMONU	2026-07-09 00:00:00.000	Kobiye Destek	350
7	2,114	Abdullah	ŞANLIURFA	2025-04-06 00:00:00.000	Kobiye Destek	350
8	2,006	Abdullah	AĞRI	2026-04-11 00:00:00.000	Kobiye Destek	350
9	1,650	Abdullah	ERZINCAN	2026-05-01 00:00:00.000	Kobiye Destek	350
10	1,429	Abdullah	AYDIN	2026-06-06 00:00:00.000	Kobiye Destek	350
11	1,282	Abdullah	BATMAN	2028-03-01 00:00:00.000	Kobiye Destek	350

APPROACH: We join the CUSTOMERS table with the TARIFFS table on TARIFF_ID, which allows us to filter directly by tariff name in the WHERE clause and ensures the query remains intact even if TARIFF_ID values change in the future. Note that the column is named NAME, not TARIFF_NAME, as it is defined that way in the TARIFFS table. Results are returned sorted alongside the customer name and city, so the output can be used directly by a customer service team.

1.2 Find the newest customer who subscribed to this tariff.

```
SELECT
    c.CUSTOMER_ID,
    c.NAME,
    c.CITY,
    c.SIGNUP_DATE,
    t.MONTHLY_FEE
FROM
    CUSTOMERS c
    JOIN TARIFFS t ON c.TARIFF_ID = t.TARIFF_ID
WHERE
    t.NAME = 'Kobiye Destek'
    AND c.SIGNUP_DATE = (
        SELECT MAX(c2.SIGNUP_DATE)
        FROM CUSTOMERS c2
        JOIN TARIFFS t2 ON c2.TARIFF_ID = t2.TARIFF_ID
        WHERE t2.NAME = 'Kobiye Destek'
    );
```

	123 CUSTOMER_ID	AZ NAME	AZ CITY	SIGNUP_DATE	123 MONTHLY_FEE
1	9,487	Murat	AKSARAY	2028-07-03 00:00:00.000	350
2	8,798	Hatice	KARABÜK	2028-07-03 00:00:00.000	350
3	7,388	Songül	SAKARYA	2028-07-03 00:00:00.000	350
4	5,418	Leyla	IĞDIR	2028-07-03 00:00:00.000	350
5	3,938	Recep	RIZE	2028-07-03 00:00:00.000	350
6	682	Yasemin	KIRIKKALE	2028-07-03 00:00:00.000	350
7	320	Meryem	DİYARBAKIR	2028-07-03 00:00:00.000	350

APPROACH: We want to find the customer with the latest SIGNUP_DATE among those subscribed to the 'Kobiye Destek' tariff. A scalar subquery calculates the maximum SIGNUP_DATE for this tariff, and the outer query returns all rows matching that date, so if multiple customers signed up on the same day, none of them are missed. A scalar subquery was preferred over NOT EXISTS because we are looking for a single matching value, making it more readable and performant.

2.1 Find the distribution of tariffs among the customers.

```
SELECT
    t.NAME AS TARIFF_NAME,
    t.MONTHLY_FEE,
    COUNT(c.CUSTOMER_ID) AS SUBSCRIBER_COUNT,
    ROUND(
        COUNT(c.CUSTOMER_ID) * 100.0
        / NULLIF(SUM(COUNT(c.CUSTOMER_ID)) OVER (), 0)
        , 2) AS RATIO_PCT
FROM
    TARIFFS t
    LEFT JOIN CUSTOMERS c ON t.TARIFF_ID = c.TARIFF_ID
GROUP BY
    t.NAME, t.MONTHLY_FEE
ORDER BY
    SUBSCRIBER_COUNT DESC;
```

	AZ TARIFF_NAME	123 MONTHLY_FEE	123 SUBSCRIBER_COUNT	123 RATIO_PCT
1	Kurumsal SMS	1,000	2,577	25.77
2	Genç Dinamik	150	2,527	25.27
3	Kobiye Destek	350	2,483	24.83
4	Çalışan GB	450	2,413	24.13

APPROACH: We use the TARIFFS table as the driving table (left side of LEFT JOIN), so that tariffs with no subscribers still appear in the results and the entire product portfolio can be evaluated in a single query. We use an analytic function (SUM ... OVER()) inside COUNT to calculate the total number of subscribers without needing a separate subquery. The RATIO_PCT column is added to simplify proportional comparisons, making it possible to evaluate small and large tariffs on the same scale.

3.1 Identify the earliest customers to sign up.

```
SELECT
    c.CUSTOMER_ID,
    c.NAME,
    c.CITY,
    c.SIGNUP_DATE,
    t.NAME AS TARIFF_NAME
FROM
    CUSTOMERS c
    JOIN TARIFFS t ON c.TARIFF_ID = t.TARIFF_ID
WHERE
    c.SIGNUP_DATE = (SELECT MIN(SIGNUP_DATE) FROM CUSTOMERS)
ORDER BY
    c.CUSTOMER_ID;
```

	123 CUSTOMER_ID	A-Z NAME	A-Z CITY	SIGNUP_DATE	A-Z TARIFF_NAME
1	875	Özlem	AFYONKARAHİSAR	2025-01-05 00:00:00.000	Çalışan GB
2	1,893	Leyla	MERSİN	2025-01-05 00:00:00.000	Genç Dinamik
3	1,917	Ayşegül	HAKKARİ	2025-01-05 00:00:00.000	Çalışan GB
4	2,394	Emre	ELAZIĞ	2025-01-05 00:00:00.000	Kobiye Destek
5	2,643	Mehmet	ESKİŞEHİR	2025-01-05 00:00:00.000	Kurumsal SMS
6	4,674	Dilek	ARTVİN	2025-01-05 00:00:00.000	Genç Dinamik
7	4,848	Osman	ŞIRNAK	2025-01-05 00:00:00.000	Çalışan GB
8	4,889	Mehmet	KOCAELİ	2025-01-05 00:00:00.000	Kurumsal SMS
9	5,109	Mustafa	AĞRI	2025-01-05 00:00:00.000	Çalışan GB
10	5,169	Yağmur	HATAY	2025-01-05 00:00:00.000	Çalışan GB
11	5,183	Hüseyin	ERZURUM	2025-01-05 00:00:00.000	Kurumsal SMS

APPROACH: As stated in the task hint, the earliest signup date does not necessarily correspond to the lowest CUSTOMER_ID, so sorting must be done by SIGNUP_DATE rather than CUSTOMER_ID. The subquery finds the minimum SIGNUP_DATE across all customers, and the outer query returns all rows that match that date. This ensures that if multiple "first customers" signed up on the same day, none of them are missed.

3.2 Find the distribution of these earliest customers across different cities, including the total count for each city.

```
SELECT
    c.CITY,
    COUNT(*) AS CUSTOMER_COUNT,
    SUM(COUNT(*)) OVER () AS TOTAL_COUNT,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS CITY_RATIO_PCT
FROM
    CUSTOMERS c
WHERE
    c.SIGNUP_DATE = (SELECT MIN(SIGNUP_DATE) FROM CUSTOMERS)
GROUP BY
    c.CITY
ORDER BY
    CUSTOMER_COUNT DESC;
```

	A-Z CITY	123 CUSTOMER_COUNT	123 TOTAL_COUNT	123 CITY_RATIO_PCT
1	ŞIRNAK	2	27	7.41
2	BATMAN	2	27	7.41
3	HAKKARİ	2	27	7.41
4	EDİRNE	1	27	3.7
5	BİNGÖL	1	27	3.7
6	ARDAHAN	1	27	3.7
7	YOZGAT	1	27	3.7
8	KAHRAMANMARAŞ	1	27	3.7
9	DÜZCE	1	27	3.7
10	BOLU	1	27	3.7

APPROACH: We combine the filter from 3.1 (minimum SIGNUP_DATE) with GROUP BY to see which cities the earliest customer cohort is concentrated in. The analytic SUM ... OVER() calculates the total row count in a single pass, so the percentage for each city is also produced within the same query. This table carries historical value as it reveals which cities the company focused on during its launch period.

4.1 Every customer has a monthly fee, and the dataset contains this month's usage values. However, an insertion error occurred, and some customers' monthly records are missing. Identify the IDs of these missing customers.

```

SELECT
  c.CUSTOMER_ID,
  c.NAME,
  c.CITY,
  t.NAME AS TARIFF_NAME,
  t.MONTHLY_FEE
FROM
  CUSTOMERS c
  JOIN TARIFFS t ON c.TARIFF_ID = t.TARIFF_ID
WHERE
  NOT EXISTS (
    SELECT 1
    FROM MONTHLY_STATS ms
    WHERE ms.CUSTOMER_ID = c.CUSTOMER_ID
  )
ORDER BY
  c.CUSTOMER_ID;

```

	123 CUSTOMER_ID	A-Z NAME	A-Z CITY	A-Z TARIFF_NAME	123 MONTHLY_FEE
1	6	Fadime	KIRŞEHİR	Kurumsal SMS	1,000
2	10	Hakan	GAZİANTEP	Genç Dinamik	150
3	31	Serkan	SİİRT	Çalışan GB	450
4	39	Yasemin	MUŞ	Genç Dinamik	150
5	45	Süleyman	KIRIKKALE	Kobiye Destek	350
6	81	Zehra	GİRESUN	Kobiye Destek	350
7	116	Emre	ADANA	Genç Dinamik	150
8	136	Zeynep	AĞRI	Kurumsal SMS	1,000
9	140	Abdullah	İZMİR	Kobiye Destek	350
10	156	Mahmut	NEVŞEHİR	Kurumsal SMS	1,000

APPROACH: Each customer should have exactly one record in the MONTHLY_STATS table; the UNIQUE constraint (UQ_STAT_CUSTOMER) guarantees this, however some customers may have no record at all due to a data loading error. We use a NOT EXISTS anti-join; compared to NOT IN, NOT IN can silently return an empty result when the subquery contains a NULL value, whereas NOT EXISTS works correctly regardless of this risk. Results are returned with a JOIN from the CUSTOMERS table, so it is also possible to see which tariff and city the missing record belongs to.

4.2 Find the distribution of these missing customers across different cities.

```
SELECT
  c.CITY,
  COUNT(*) AS MISSING_COUNT,
  SUM(COUNT(*)) OVER () AS TOTAL_MISSING,
  ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS CITY_RATIO_PCT
FROM
  CUSTOMERS c
WHERE
  NOT EXISTS (
    SELECT 1
    FROM MONTHLY_STATS ms
    WHERE ms.CUSTOMER_ID = c.CUSTOMER_ID
  )
GROUP BY
  c.CITY
ORDER BY
  MISSING_COUNT DESC;
```

	AZ CITY	123 MISSING_COUNT	123 TOTAL_MISSING	123 CITY_RATIO_PCT
1	OSMANIYE	3	50	6
2	BİTLİS	2	50	4
3	İZMİR	2	50	4
4	ORDU	2	50	4
5	DENİZLİ	2	50	4
6	MUŞ	2	50	4
7	NEVŞEHİR	2	50	4
8	KAYSERİ	2	50	4
9	SİVAS	2	50	4
10	KIRIKKALE	2	50	4

APPROACH: We wrap the NOT EXISTS filter from 4.1 with GROUP BY to calculate how many customers have missing records on a city basis. This view can be used to determine whether the missing records originate from a specific region or are randomly distributed; if there is a regional system failure, city concentration will reveal it. The percentage ratio provides a fair scale when comparing cities of different sizes.

5.1 Find the customers who have used at least 75% of their data limit.

```
SELECT
  c.CUSTOMER_ID,
  c.NAME,
  c.CITY,
  t.NAME AS TARIFF_NAME,
  t.DATA_LIMIT,
  ms.DATA_USAGE,
  ROUND(ms.DATA_USAGE / NULLIF(t.DATA_LIMIT, 0) * 100, 2) AS DATA_USAGE_PCT
FROM
  CUSTOMERS c
  JOIN TARIFFS t ON c.TARIFF_ID = t.TARIFF_ID
  JOIN MONTHLY_STATS ms ON ms.CUSTOMER_ID = c.CUSTOMER_ID
WHERE
  t.DATA_LIMIT > 0
  AND ms.DATA_USAGE / NULLIF(t.DATA_LIMIT, 0) >= 0.75
ORDER BY
  DATA_USAGE_PCT DESC;
```

	123 CUSTOMER_ID	AZ NAME	AZ CITY	AZ TARIFF_NAME	123 DATA_LIMIT	123 DATA_USAGE	123 DATA_USAGE_PCT
1	5,623	Meryem	MALATYA	Kobiye Destek	20,480	20,476.27	99.98
2	8,825	Ayşe	ISTANBUL	Kobiye Destek	20,480	20,476.31	99.98
3	311	Fatma	KARAMAN	Çalışan GB	20,480	20,476.18	99.98
4	2,770	Ahmet	ŞANLIURFA	Kobiye Destek	20,480	20,474.77	99.97
5	8,960	Yağmur	ŞIRNAK	Çalışan GB	20,480	20,466.81	99.94
6	666	Fadime	YALOVA	Genç Dinamik	10,240	10,234.05	99.94
7	6,924	Halil	ÇANKIRI	Çalışan GB	20,480	20,465.87	99.93
8	4,749	Aynur	BARTIN	Kobiye Destek	20,480	20,460.43	99.9
9	2,655	Tuğba	BİLECİK	Genç Dinamik	10,240	10,229.96	99.9

APPROACH: We join the CUSTOMERS, TARIFFS and MONTHLY_STATS tables to compare each customer's data consumption (DATA_USAGE) against their tariff's DATA_LIMIT value. The NULLIF(t.DATA_LIMIT, 0) expression prevents a division by zero error for tariffs with a zero data limit (such as the 'Kurumsal SMS' package). Since the threshold is defined as a ratio (≥ 0.75) rather than a fixed MB value, the query works consistently across all tariffs.

5.2 Identify the customers who have completely exhausted all of their package limits (data, minutes, and SMS).

```
SELECT
  c.CUSTOMER_ID,
  c.NAME,
  c.CITY,
  t.NAME AS TARIFF_NAME,
  ms.DATA_USAGE, t.DATA_LIMIT,
  ms.MINUTE_USAGE, t.MINUTE_LIMIT,
  ms.SMS_USAGE, t.SMS_LIMIT
FROM
  CUSTOMERS c
  JOIN TARIFFS t ON c.TARIFF_ID = t.TARIFF_ID
  JOIN MONTHLY_STATS ms ON ms.CUSTOMER_ID = c.CUSTOMER_ID
WHERE
  ms.DATA_USAGE >= t.DATA_LIMIT
  AND ms.MINUTE_USAGE >= t.MINUTE_LIMIT
  AND ms.SMS_USAGE >= t.SMS_LIMIT
ORDER BY
  c.CUSTOMER_ID;
```

123	CUSTOMER_ID	AZ	NAME	AZ	CITY	AZ	TARIFF_NAME	123	DATA_USAGE	123	DATA_LIMIT	123	MINUTE_USAGE	123

APPROACH: The conditions are combined with AND since all three resources must be exhausted simultaneously, which identifies the most constrained customer profile. The >= operator is used to account for the possibility that usage may occasionally exceed the limit due to rounding or delayed throttling in the source system. There are no customers in the current dataset that satisfy this condition; the query is working correctly and an empty result is the expected outcome.

6.1 Find the customers who have unpaid fees.

```
SELECT
  c.CUSTOMER_ID,
  c.NAME,
  c.CITY,
  ms.PAYMENT_STATUS,
  t.NAME AS TARIFF_NAME,
  t.MONTHLY_FEE
FROM
  CUSTOMERS c
  JOIN TARIFFS t ON c.TARIFF_ID = t.TARIFF_ID
  JOIN MONTHLY_STATS ms ON ms.CUSTOMER_ID = c.CUSTOMER_ID
WHERE
  ms.PAYMENT_STATUS IN ('LATE', 'UNPAID')
ORDER BY
  ms.PAYMENT_STATUS, t.MONTHLY_FEE DESC;
```

123	CUSTOMER_ID	AZ	NAME	AZ	CITY	AZ	PAYMENT_STATUS	AZ	TARIFF_NAME	123	MONTHLY_FEE
1	8,691		Hacer		KARS		LATE		Kurumsal SMS		1,000
2	8,707		Mehmet		ŞIRNAK		LATE		Kurumsal SMS		1,000
3	8,708		Zehra		İZMİR		LATE		Kurumsal SMS		1,000
4	8,733		Abdullah		SAMSUN		LATE		Kurumsal SMS		1,000
5	8,743		Zehra		DİYARBAKIR		LATE		Kurumsal SMS		1,000
6	8,758		Emre		ÇANAKKALE		LATE		Kurumsal SMS		1,000
7	8,768		Ayşegül		HAKKARİ		LATE		Kurumsal SMS		1,000
8	8,471		Merve		ESKİŞEHİR		LATE		Kurumsal SMS		1,000
9	8,477		Hacer		TUNCELİ		LATE		Kurumsal SMS		1,000
10	8,488		Melek		BURSA		LATE		Kurumsal SMS		1,000
11	8,490		Hacer		SAMSUN		LATE		Kurumsal SMS		1,000

APPROACH: Values other than 'PAID' in the PAYMENT_STATUS column (LATE, UNPAID,) indicate a payment problem. We handle these statuses in a single condition using the IN operator; if new status codes are added in the future, only this list needs to be updated. Tariff and monthly fee information is also included in the results, so the output can be

used directly as an accounts receivable report containing debt amounts. Results are sorted by PAYMENT_STATUS so that overdue (LATE) customers appear at the top.

6.2 Find the distribution of all payment statuses across the different tariffs.

```
SELECT
    t.NAME AS TARIFF_NAME,
    t.MONTHLY_FEE,
    COUNT(ms.ID) AS TOTAL_CUSTOMERS,
    COUNT(CASE WHEN ms.PAYMENT_STATUS = 'PAID' THEN 1 END) AS PAID_COUNT,
    COUNT(CASE WHEN ms.PAYMENT_STATUS = 'LATE' THEN 1 END) AS LATE_COUNT,
    COUNT(CASE WHEN ms.PAYMENT_STATUS = 'UNPAID' THEN 1 END) AS UNPAID_COUNT,
    ROUND(
        COUNT(CASE WHEN ms.PAYMENT_STATUS != 'PAID' THEN 1 END)
        * 100.0 / NULLIF(COUNT(ms.ID), 0)
    , 2) AS DELINQUENCY_RATE_PCT
FROM
    TARIFFS t
    LEFT JOIN CUSTOMERS c ON t.TARIFF_ID = c.TARIFF_ID
    LEFT JOIN MONTHLY_STATS ms ON c.CUSTOMER_ID = ms.CUSTOMER_ID
GROUP BY
    t.NAME, t.MONTHLY_FEE
ORDER BY
    DELINQUENCY_RATE_PCT DESC NULLS LAST;
```

	AZ TARIFF_NAME	123 MONTHLY_FEE	123 TOTAL_CUSTOMERS	123 PAID_COUNT	123 LATE_COUNT	123 UNPAID_COUNT	123 DELINQUENCY_RATE_PCT
1	Kobiye Destek	350	2,471	1,719	392	360	30.43
2	Kurumsal SMS	1,000	2,567	1,796	368	403	30.04
3	Çalışan GB	450	2,396	1,692	365	339	29.38
4	Genç Dinamik	150	2,516	1,792	372	352	28.78

APPROACH: We use conditional aggregation (COUNT inside CASE WHEN) to display the number of PAID / LATE / UNPAID subscribers per tariff side by side in a single query, which is more portable than the PIVOT syntax. The DELINQUENCY_RATE_PCT column shows the ratio of customers with payment problems to the total number of subscribers on that tariff, quickly revealing which package carries the highest financial risk. LEFT JOIN is used to ensure that tariffs with no subscribers also appear in the results.

BONUS 1: Monthly Revenue at Risk by Tariff

```
SELECT
    t.NAME AS TARIFF_NAME,
    t.MONTHLY_FEE,
    COUNT(ms.ID) AS DELINQUENT_CUSTOMERS,
    COUNT(ms.ID) * t.MONTHLY_FEE AS REVENUE_AT_RISK_TRY
FROM
    CUSTOMERS c
    JOIN TARIFFS t ON c.TARIFF_ID = t.TARIFF_ID
    JOIN MONTHLY_STATS ms ON ms.CUSTOMER_ID = c.CUSTOMER_ID
WHERE
    ms.PAYMENT_STATUS IN ('LATE', 'UNPAID')
GROUP BY
    t.NAME, t.MONTHLY_FEE
ORDER BY
    REVENUE_AT_RISK_TRY DESC;
```


	AZ TARIFF_NAME	123 MONTHLY_FEE	123 DELINQUENT_CUSTOMERS	123 REVENUE_AT_RISK_TRY
1	Kurumsal SMS	1,000	771	771,000
2	Çalışan GB	450	704	316,800
3	Kobiye Destek	350	752	263,200
4	Genç Dinamik	150	724	108,600

APPROACH: Calculates the total monthly revenue at risk in TRY for customers with payment problems, broken down by tariff. Presenting a concrete monetary value rather than an abstract number makes it easier for the finance team to prioritize actions.

BONUS 2: High Churn Risk Customers (High Usage + Payment Problem)

```
SELECT
    c.CUSTOMER_ID,
    c.NAME,
    c.CITY,
    ms.PAYMENT_STATUS,
    t.NAME AS TARIFF_NAME,
    ROUND(ms.DATA_USAGE / NULLIF(t.DATA_LIMIT,0) * 100, 1) AS DATA_USAGE_PCT,
    t.MONTHLY_FEE
FROM
    CUSTOMERS c
    JOIN TARIFFS t ON c.TARIFF_ID = t.TARIFF_ID
    JOIN MONTHLY_STATS ms ON ms.CUSTOMER_ID = c.CUSTOMER_ID
WHERE
    t.DATA_LIMIT > 0
    AND ms.DATA_USAGE / NULLIF(t.DATA_LIMIT, 0) >= 0.90
    AND ms.PAYMENT_STATUS IN ('LATE', 'UNPAID')
ORDER BY
    DATA_USAGE_PCT DESC;
```

	123 CUSTOMER_ID	AZ NAME	AZ CITY	AZ PAYMENT_STATUS	AZ TARIFF_NAME	123 DATA_USAGE_PCT	123 MONTHLY_FEE
1	311	Fatma	KARAMAN	UNPAID	Çalışan GB	100	
2	8,960	Yağmur	ŞIRNAK	UNPAID	Çalışan GB	99.9	
3	2,655	Tuğba	BİLECİK	UNPAID	Genç Dinamik	99.9	
4	4,001	Büşra	İZMİR	UNPAID	Genç Dinamik	99.9	
5	6,871	Ayşe	BURSA	UNPAID	Genç Dinamik	99.9	
6	7,230	Havva	ÇORUM	LATE	Çalışan GB	99.8	
7	706	Burak	AKSARAY	LATE	Genç Dinamik	99.7	
8	4,706	Murat	OSMANİYE	LATE	Kobiye Destek	99.6	

APPROACH: Identifies customers whose data usage is $\geq 90\%$ AND who have a payment problem. This combination represents the "heavy user + debtor" profile and is the highest priority intervention group in terms of both revenue loss and network quality.

BONUS 3: Total Revenue Summary by Tariff

```
SELECT
    t.NAME AS TARIFF_NAME,
    t.MONTHLY_FEE,
    COUNT(c.CUSTOMER_ID) AS TOTAL_SUBSCRIBERS,
    COUNT(c.CUSTOMER_ID) * t.MONTHLY_FEE AS TOTAL_BILLED_TRY,
    COUNT(CASE WHEN ms.PAYMENT_STATUS = 'PAID' THEN 1 END)
    * t.MONTHLY_FEE AS COLLECTED_TRY,
    (COUNT(c.CUSTOMER_ID) -
    COUNT(CASE WHEN ms.PAYMENT_STATUS = 'PAID' THEN 1 END))
    * t.MONTHLY_FEE AS OUTSTANDING_TRY
FROM
    TARIFFS t
    LEFT JOIN CUSTOMERS c ON t.TARIFF_ID = c.TARIFF_ID
    LEFT JOIN MONTHLY_STATS ms ON c.CUSTOMER_ID = ms.CUSTOMER_ID
GROUP BY
    t.NAME, t.MONTHLY_FEE
ORDER BY
    OUTSTANDING_TRY DESC NULLS LAST;
```

	AZ TARIFF_NAME	123 MONTHLY_FEE	123 TOTAL_SUBSCRIBERS	123 TOTAL_BILLED_TRY	123 COLLECTED_TRY	123 OUTSTANDING_TRY
1	Kurumsal SMS	1,000	2,577	2,577,000	1,796,000	781,00
2	Çalışan GB	450	2,413	1,085,850	761,400	324,45
3	Kobiye Destek	350	2,483	869,050	601,650	267,40
4	Genç Dinamik	150	2,527	379,050	268,800	110,25

APPROACH: Displays the total billed revenue, collected revenue and outstanding (at risk) amount for each tariff side by side. Can be used for end-of-month financial reconciliation and CFO reporting.